SCANet: Correcting LEGO Assembly Errors with Self-Correct Assembly Network

Yuxuan Wan[1]* Kaichen Zhou[2]* Jinhong Chen[2] Hao Dong[2]#
[1]School of Computer Science and Engineering, Southeast University
[2]School of Computer Science, Peking University

***Abstract*— Autonomous assembly in robotics and 3D vision presents significant challenges, particularly in ensuring assembly correctness. Presently, predominant methods such as MEPNet focus on assembling components based on manually provided images. However, these approaches often fall short in achieving satisfactory results for tasks requiring long-term planning. Concurrently, we observe that integrating a self-correction module can partially alleviate such issues. Motivated by this concern, we introduce the *single-step assembly error correction task*, which involves identifying and rectifying misassembled components. To support research in this area, we present the LEGO Error Correction Assembly Dataset (LEGO-ECA), comprising manual images for assembly steps and instances of assembly failures. Additionally, we propose the Self-Correct Assembly Network (SCANet), a novel method to address this task. SCANet treats assembled components as queries, determining their correctness in manual images and providing corrections when necessary. Finally, we utilize SCANet to correct the assembly results of MEPNet. Experimental results demonstrate that SCANet can identify and correct MEPNet's misassembled results, significantly improving the correctness of assembly. Our code and dataset are available at https://github.com/Yaser-wyx/SCANet.**

I. Introduction

For a considerable period, the endeavor to enable robots for autonomous assembly of parts based on human-provided assembly manuals, such as block worlds [1], LEGO models [2], [3], and furniture [4], [5], has been a longstanding pursuit. However, despite the apparent simplicity of such tasks for humans, they pose formidable challenges for robots. Achieving this goal demands extensive engineering efforts and theoretical breakthroughs. Moreover, while humans are prone to errors during part assembly, potentially misplacing various components, they possess the capability to detect and rectify these errors during subsequent assembly processes. Nevertheless, existing research works [3], [6], [7], [8], [9], [10] have largely overlooked this aspect, expecting robots to execute flawless assembly sequences from start to finish in a single attempt. This expectation proves unrealistic and overly stringent for robots. As assembly progresses, current methods tend to accumulate an increasing number of assembly errors, deviating from the intended sequence. These errors lead to larger discrepancies between the final assembly result and the assembly manual, as illustrated in Figure 1. To address this critical research gap, we propose a new task, namely the *"Single-Step Assembly Error Correction Task"*.

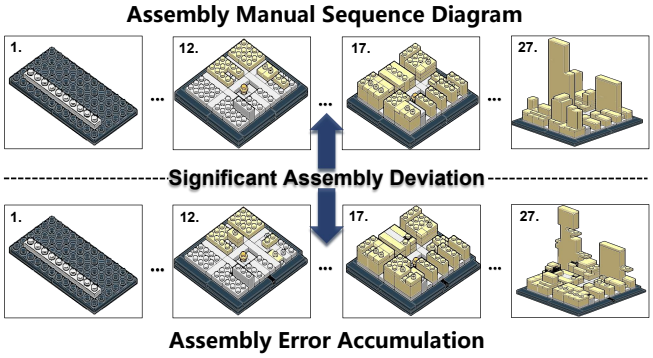


Fig. 1: We observed that as the assembly process progresses, assembly errors accumulate, leading to larger discrepancies between the final assembly result and the assembly manual.

This task involves identifying components with 6D pose errors in the current assembly step and correcting them. This task presents two new challenges. Firstly, *how to identify misassembled components in the current assembly result?* This requires the ability to discern the current pose of each assembled component, forming the foundational step for conducting assembly error correction. Focusing solely on misassembled components helps mitigate the risk of adjusting correctly assembled components into incorrect pose, thereby reducing the Misplacement Rate (MPR, defined in Section V-C). Secondly, *how to correct identified misassembled components?* This involves establishing the relationship between the misassembled component and its correct pose as described in the assembly manual. By doing so, the misassembled component can be adjusted to its correct pose, thus rectifying the assembly error.

To address these challenges, we first constructed a dataset specifically tailored for the single-step assembly correction task, called the LEGO Error Correction Assembly Dataset (LEGO-ECA), which is based on the Synthetic LEGO Dataset [3]. This dataset comprises 1,429 LEGO assembly manuals, each containing 2D illustrations of each assembly step along with various possible assembly failure scenarios. Further details about the dataset are provided in Section III.

Continuing on, addressing the assembly error correction problem, a natural approach is to compare existing assembly

* Equal contribution
# Corresponding author

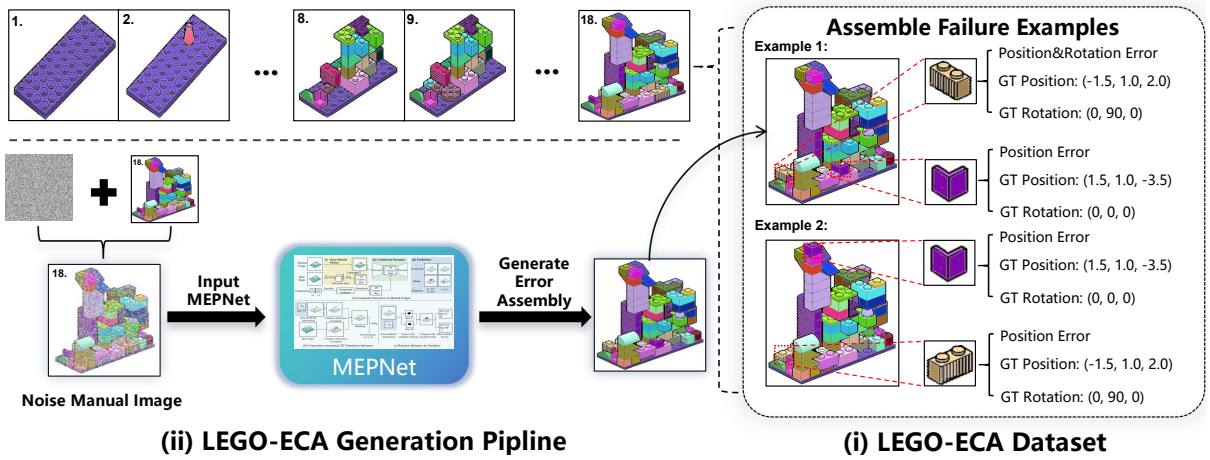


Fig. 2: The LEGO-ECA dataset, designed specifically for the single-step assembly error correction task, is presented. (i) Exemplars illustrate assembly manual sequence diagram, instances of component assembly errors, error types, and correct poses. (ii) The construction process outlines how erroneous assembly examples were generated in the LEGO-ECA dataset.

results with assembly manuals, identifying differences to determine which components are misassembled in the current step and their correct poses. This approach mirrors human error correction methods. Therefore, we transformed the challenging correction problem into a relatively straightforward query task. Building upon this idea, we introduce a novel architecture called the Self-Correct Assembly Network (SCANet). The core concept of this architecture is to treat each assembled component as a query, querying its assembly status in the manual image, and providing the correct pose when errors occur. More details of SCANet are provided in Section IV.

To the best of our knowledge, this represents first attempt to explore and address the single-step assembly error correction task. We constructed the LEGO-ECA dataset and proposed the SCANet architecture to address this task. Ultimately, we utilize the SCANet to correct the assembly results of MEPNet [3], and experimental results demonstrate that SCANet can identify and correct MEPNet's misassembled component errors, significantly improving component assembly accuracy.

II. Related Work

A. Part Assembly Datasets

In recent decades, numerous 3D part datasets have emerged. Previous works have primarily focused on part assembly [11], [12], [13], [14], [15]. The PartNet dataset [13] consists of 573,585 part instances across 26,671 3D models covering 24 object categories. This dataset has facilitated various tasks such as shape analysis, dynamic 3D scene modeling and simulation, affordance analysis, and more. Wang *et al.* [4] introduced IKEA-Manual, a dataset with annotated IKEA objects and assembly manuals. It comprises realistic 3D assembly objects paired with human-designed manuals, parsed in a structured format. Furthermore, the Synthetic LEGO dataset [3] comprises 9,000 LEGO manuals, each presenting a step-by-step instruction sequence for adding new components to an existing LEGO shape. Despite the advancements in prior works, existing datasets have overlooked inherent errors in complex assembly processes. In this paper, we aim to bridge this gap by introducing the LEGO Error Correction Assembly (LEGO-ECA) Dataset, which serves as a dataset for assembly error correction task. It is constructed based on the Synthetic LEGO Dataset utilized in MEPNet [3].

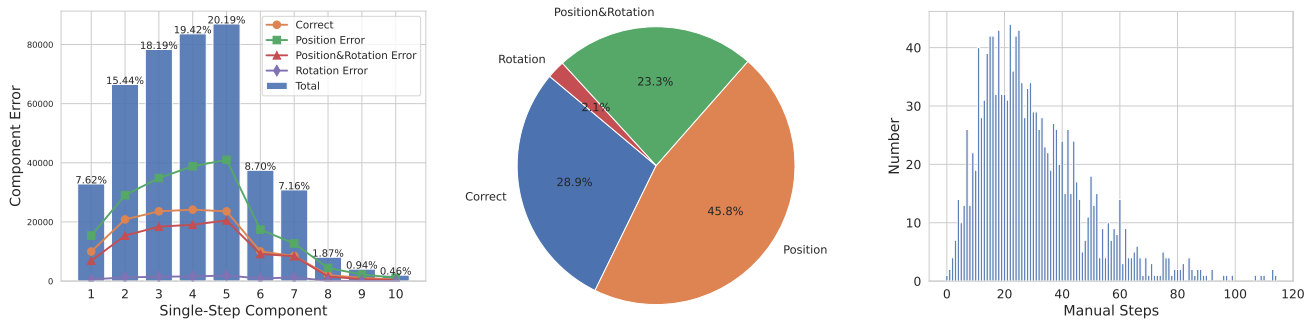
B. 3D Part Assembly

Recent advancements [3], [6], [7], [8], [9], [16], [17], [18], [19] in assembly guidance leverage cutting-edge technologies to improve efficiency and accuracy. Huang *et al.* [6] pioneers a dynamic graph learning framework, predicting part poses within point clouds by reasoning over their relations using dynamic modules. Li *et al.* [7] proposes a two-module pipeline for single-image-guided 3D part assembly, emphasizing accurate pose prediction and relationship constraints through 2D-3D correspondences and graph message-passing. Additionally, Wang *et al.* [3] introduces MEPNet, employing neural keypoint detection and 2D-3D projection algorithms for reconstructing assembly steps from manual images. These innovative approaches signify a paradigm shift in assembly processes, promising enhanced precision and automation. While the mentioned methods [7], [6], [3] focus primarily on how to assemble components, they overlook the potential for errors in this process. Our work takes a pivotal turn by not only addressing the act of assembly but also delving deeper into the crucial aspect of ensuring its accuracy. We go beyond mere assembly guidance to incorporate mechanisms for error detection and correction.

Thus, our approach marks a significant advancement, shifting the emphasis from solely executing assembly tasks to actively verifying and rectifying any inaccuracies in the assembled components.

III. LEGO Error Correction Assembly Dataset

The LEGO Error Correction Assembly Dataset (LEGO-ECA) is a dataset constructed for the single-step assembly correction task, based on the Synthetic LEGO dataset used



(a) Error Components in Single-Step Assembly (b) Distribution of Different Error Components (c) Manuals with different Numbers of Steps

Fig. 3: LEGO-ECA Dataset Statistics. (a) Proportions of different types of incorrectly assembled components in single-step assembly. In single-step assembly, involving typically 2 to 5 components, positional errors are the most prevalent. Rotational errors frequently accompany positional misalignment and tend to occur concurrently with positional errors, resulting in fewer instances of isolated rotational errors. (b) Distribution of different types of incorrectly assembled components across the entire dataset. Positional errors are predominant, while rotational errors are the least frequent. Correct components and those with both positional and rotational errors roughly occupy similar proportions. (c) Statistics of the number of manuals with different numbers of steps. The majority of manuals have step counts ranging from 15 to 40.

in MEPNet [3]. The construction process and composition of the dataset are illustrated in Figure 2. It comprises 1,429 LEGO assembly manuals, each containing 2D illustrations of each assembly step and various possible assembly failure examples. For each assembly failure example, as illustrated in Figure 2(i), we provide a 2D rendering of the failed assembly and the corresponding 3D shape. Additionally, we annotate the error types of each incorrectly assembled component and the correct assembly pose.

To construct the LEGO-ECA dataset, as shown in Figure 2(ii), we randomly selected 1,429 assembly manuals from the training set of the Synthetic LEGO Dataset. Subsequently, we inputted each step into MEPNet for component assembly. To simulate incorrect assembly samples more effectively and consistently, we preprocessed the manual images before inputting them into MEPNet. This involved adding random Gaussian noise to interfere with the assembly process, thereby inducing MEPNet to output incorrectly assembled examples. For each assembly step, we applied 3 to 5 random Gaussian noise injections. Ultimately, we constructed a dataset containing approximately 120,000 instances of incorrectly assembled examples. We conducted statistical analysis on the LEGO-ECA dataset, and the relevant results are shown in Figure 3. Figure 3a depicts the ratio of components in different states to the total number of components used in single-step assembly. Figure 3b presents the ratio of different error types of components to all components. Figure 3c illustrates the distribution of the number of manuals with different numbers of steps.

IV. Self-Correct Assembly Network

A. Problem Formulation

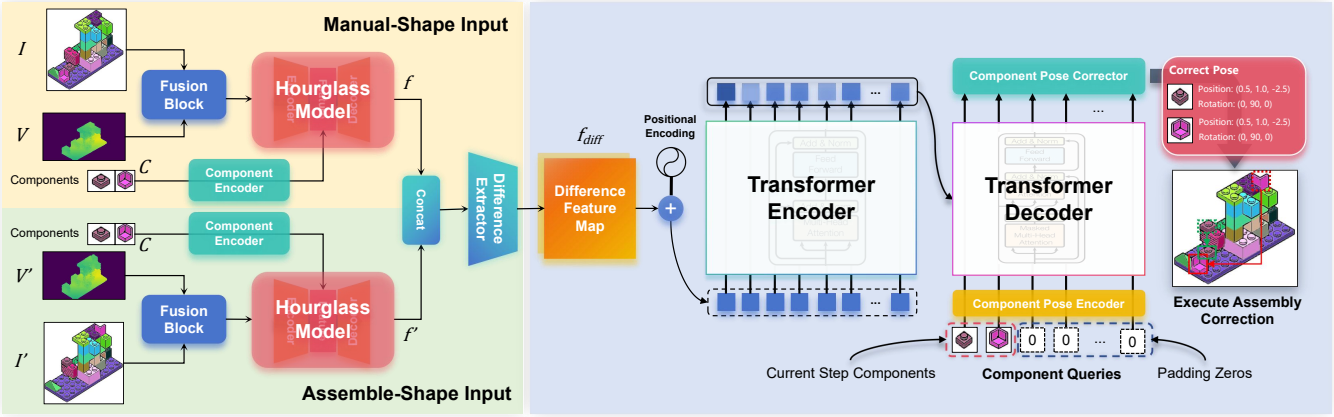
In this context, we define the *Single-Step Assembly Correction Task*. Firstly, the prerequisite for performing the assembly correction task is completing the component assembly task. Therefore, we denote all prerequisites needed for the assembly task as C_a, which include 2D manual images I, the initial shapes before assembly V, and all components used $C = \{c_1, c_2, \ldots, c_N\}$. Additionally, assembly correction requires the component poses after assembly $P_i = \{(R_i, T_i) | R_i \in \mathbb{R}^{3\times3}, T_i \in \mathbb{R}^3, i = 1, 2, \ldots, N\}$ obtained by the model M, the resulting shape after assembly V', and its corresponding 2D rendered image I'. The objective of our assembly correction task is as follows: based on C_a, given V' and I' obtained after assembly by the assembly model M, predict the status S_i of each component's pose P_i, and correct each S_i predicted as an erroneous component pose P_i^{error} to the correct component pose $P_i^{correct}$.

B. Architecture Overview

Drawing from Figure 4, the architecture of our proposed model, SCANet, shares similarities with DETR [20] and can be broadly divided into two modules. The first module, depicted in Figure 4(i), is the convolutional neural network backbone. This backbone network plays a pivotal role in extracting the difference feature f_{diff} between assembly manual image and assembly result. It takes images I and I', as well as the 3D shape V and V', and components C as inputs. The second module, illustrated in Figure 4(ii), is the assembly correction module. It is designed to determine whether components are assembled correctly and correct any misassembled components. This module takes as inputs the features f_{diff} outputted by the backbone network, the components C_i used in the current step, and the current pose P_i, along with the 2D image I'_C of each component. Subsequently, it outputs the status information S_i of each component, indicating whether it is correctly assembled, along with the 6D pose information P'_i of each component after correction.

C. Convolutional Neural Network Backbone

As illustrated in Figure 4(i), the convolutional neural network backbone of SCANet shares a similar structure to



(i) Convolutional Neural Network Backbone (ii) Assemble Correction Module

Fig. 4: SCANet consists of two modules. (i) a convolutional neural network backbone, comprising a fusion block, Hourglass model, and assembly difference extractor, which extracts differential features between manual images and assembly results; and (ii) an assembly correction module, the core of SCANet, consisting of three parts: a component pose encoder, transformer network, and component pose corrector, which outputs the final corrected component pose information.

MEPNet [3], incorporating the image-shape fusion block and the Hourglass model [21]. However, SCANet diverges from MEPNet with its inclusion of two input branches: the Manual-Shape branch and the Assemble-Shape branch. These branches share identical network structures and weight parameters. Additionally, SCANet integrates an assembly difference extractor to capture the disparity between the assembly result and the assembly manual. Specifically, the image I, 3D shape V, and components C are fed into the Manual-Shape branch, yielding feature f after passing through the fusion block and Hourglass model. Similarly, the image I', 3D shape V', and components C are input to the Assemble-Shape branch, generating feature f'. Both features possess dimensions of $\mathbb{R}^{C_1 \times H_1 \times W_1}$, where $H_1, W_1 = \frac{H_{\text{img}}}{4}, \frac{W_{\text{img}}}{4}$, and $C_1 = 128$. Subsequently, by concatenating features f and f' along the channel dimension, a new feature f_{cat} is formed with dimensions $\mathbb{R}^{2C_1 \times H_1 \times W_1}$. Finally, f_{cat} undergoes processing by the assembly difference extractor to derive the difference feature f_{diff} with dimensions $\mathbb{R}^{C_2 \times H \times W}$, where $C_2 = 256$, and $H, W = \frac{H_{\text{img}}}{16}, \frac{W_{\text{img}}}{16}$. The assembly difference extractor comprises two residual blocks.

D. Assembly Correction Module

As depicted in Figure 4(ii), the assembly correction module of SCANet serves as the core module of the network and comprises three parts: the component pose encoder, transformer network [22], and component pose corrector. The inputs to the entire module include the difference feature f_{diff} obtained from the backbone, the 3D voxel data C of the components, the 2D image I'_C of the components after assembly, and the pose information P of the components. The outputs include the status information S of the components and the corrected components' pose P'.

Component Pose Encoder: As illustrated in Figure 5, the component pose encoder comprises three sub-encoders: the 3D voxel encoder, the 6D pose encoder, and the 2D image encoder.

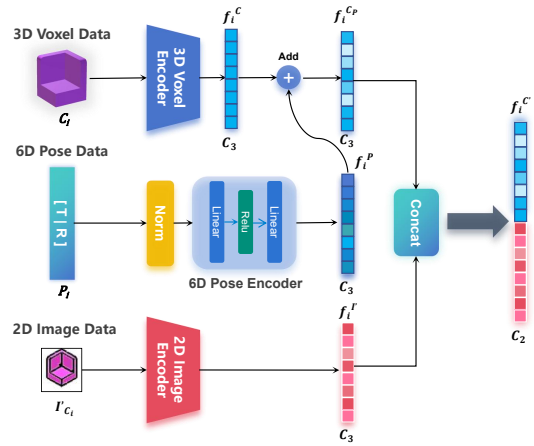


Fig. 5: The component pose encoder consists of three sub-encoders: a 3D voxel encoder, a 6D pose encoder, and a 2D image encoder. The 3D voxel encoder encodes the component's 3D voxel data into a one-dimensional vector, which is then combined with the output of the 6D pose encoder. This combined vector is concatenated with the output of the 2D image encoder, resulting in a component feature that integrates 6D pose information, 2D image information, and 3D shape information simultaneously.

Firstly, the 3D voxel encoder adopts a structure similar to the component encoder in MEPNet. It consists of multiple 3D Convolution-BatchNorm-ReLU layers, followed by an average pooling layer and a flatten layer. This process encodes the 3D voxel data into a one-dimensional vector feature represented as $f_i^C \in \mathbb{R}^{C_3}$.

Next, the 6D pose encoder comprises two fully connected layers. It takes the component's 6D pose information $P = [T|R]$, where $P \in \mathbb{R}^6$. $T \in \mathbb{R}^3$ represents the component's coordinates in 3D space, and $R \in \mathbb{R}^3$ denotes the Euler angle representation of the component. The output is the encoded 6D pose feature $f_i^P \in \mathbb{R}^{C_3}$. Before feeding into the 6D pose encoder, we normalize each component's T_i and R_i separately

according to the following formula:

$$\begin{cases} T_i^x, R_i^x = \frac{T_i^x - T_{min}^x}{T_{max}^x - T_{min}^x}, \frac{R_i^x - R_{min}^x}{R_{max}^x - R_{min}^x} \\ T_i^y, R_i^y = \frac{T_i^y - T_{min}^y}{T_{max}^y - T_{min}^y}, \frac{R_i^y - R_{min}^y}{R_{max}^y - R_{min}^y} \\ T_i^z, R_i^z = \frac{T_i^z - T_{min}^z}{T_{max}^z - T_{min}^z}, \frac{R_i^z - R_{min}^z}{R_{max}^z - R_{min}^z} \end{cases} \tag{1}$$

We then add f_i^P and f_i^C to obtain a component feature $f_i^{C_P} \in \mathbb{R}^{C_3}$, which encodes the 6D pose information. Subsequently, for the 2D image encoder, we employ the ResNet network [23] structure, taking the 2D image data I'_{C_i} of each component after assembly as input, and producing the encoded component image feature $f_i^{I'} \in \mathbb{R}^{C_3}$. Here, we modify the output dimensionality of the ResNet. Finally, we concatenate $f_i^{C_P}$ and $f_i^{I'}$ along the channel dimension, yielding a component feature $f_i^{C'} \in \mathbb{R}^{C_2}$ that integrates both the 6D pose information and the 2D image information simultaneously.

It's important to highlight that encoding not only the 6D pose information, but also the 2D image information for each component serves a crucial purpose. During the correction process, we observed that if multiple components of the same shape are present in current assembly step, differing only in their 6D poses, SCANet may inadvertently correct them all to the same pose. However, by incorporating image information, which includes color data of the components, this issue can be mitigated to some extent.

Transformer Network: The encoder and decoder of the transformer model in our approach closely resemble those in DETR [20], following the standard transformer architecture. However, there are slight differences in our decoder. Firstly, while DETR uses learned object query embeddings, our component query embeddings are obtained by encoding through the component pose encoder. Secondly, in DETR, the number of object queries input into the decoder is fixed, whereas the length sequence of component queries we input varies. For each batch, the length of queries follows the maximum number of component queries in that batch, with any remaining samples having fewer queries padded with zero vectors.

Component Pose Corrector: The component pose corrector comprises three small fully connected neural networks with distinct structures: the component status classifier, the component 3D position correction head, and the component rotation correction head.

1) **Component Status Classifier:** This classifier consists of two fully connected layers. Its output, a feature vector $f_{1d} \in \mathbb{R}^{C_4}$, where $C_4 = 4$, encodes four possible component statuses: correctly assembled, position error, rotation error, or both position and rotation errors.
2) **Component 3D Position Correction Head:** We adopt the structure of SimCC [24], treating each component's 3D coordinates as a classification task in three directions (x, y, z). Specifically, after processing through this head, it will output a tuple consisting of three 1D feature maps (f_x, f_y, f_z). Additionally, the size of each feature map is consistent with the size of the voxel grid in each direction.
3) **Component Rotation Correction Head:** Following a similar approach to MEPNet, we treat different rotation angles (0°, 90°, 180°, 270°) as a classification task. However, we simplify the treatment of rotational symmetry. While MEPNet considers rotational symmetry in the classification task, we do not consider it when outputting the rotation correction result. Instead, symmetry of the component is considered during specific correction. For example, if a component is symmetry at 0° and 180°, its rotation is considered as 0°.

E. Training and Loss

We train SCANet on the LEGO-ECA dataset, which provides component status, 3D position, and rotation information. The network is trained end-to-end using gradient descent. Our objective function is defined as:

$$L = \alpha \cdot L_{position} + \beta \cdot L_{rotation} + \gamma \cdot L_{status} \tag{2}$$

Here, $L_{position}$, $L_{rotation}$, and L_{status} are all cross-entropy losses. Unlike MPNet, which employs various loss functions for training, we simplify all tasks into classification tasks, making the overall network easier to train. Additionally, we adopt auxiliary loss settings inspired by DETR [20] to further accelerate network convergence. Specifically, we provide the output of each layer of the transformer decoder to the component pose corrector, calculate the loss based on the corrector's output, and then take the mean loss of all layers.

V. Experiments

A. Implementation Details

SCANet trains for 100 epochs on the training set, utilizing the AdamW optimizer with a learning rate set to 1e-4. It employs a batch size of 8 and applies gradient accumulation with an accumulation step of 4. Based on empirical observations, the hyperparameters α and β of the loss function were set to 1 each, while γ is set to 0.5. The image input size remained consistent with MEPNet [3] at 512×512. Lastly, both the transformer network and the ResNet [23] network used in SCANet employed their respective pretrained models.

B. Dataset

We first randomly partition each manual in the LEGO-ECA dataset into 80% for training and 20% for validation. During training and validation, we ensure that only components with errors in the current step are considered. In other words, components not belonging to the current step are correctly assembled. Subsequently, we randomly select 10% (150 manuals) of the LEGO-ECA dataset for assembly error correction testing. During assembly error correction testing, we adopt a setwise setting similar to MEPNet, wherein we correct the assembly results of the current step and use the corrected results as input for the next step of assembly. However, since our correction process cannot guarantee 100%

Dataset	Method	Pose Acc ↑ (%)		MTC ↓ (%)	CR ↑ (%)	MPR ↓ (%)	CD ↓
		Component	Step				
LEGO-ECA	MEPNet [3]	38.58	28.66	66.46	–	–	28.98
	SCANet	**49.31**$^{\uparrow 10.73}$	**36.96**$^{\uparrow 8.30}$	**58.07**$^{\downarrow 8.33}$	**19.31**	**6.35**	**12.61**$^{\downarrow 16.37}$
Synthetic-LEGO	MEPNet [3]	33.70	24.94	69.80	–	–	40.02
	SCANet	**38.29**$^{\uparrow 5.22}$	**29.24**$^{\uparrow 4.30}$	**64.92**$^{\downarrow 4.88}$	**9.02**	**2.47**	**24.08**$^{\downarrow 15.94}$

TABLE I: Experimental results of assembly correction on LEGO-ECA and Synthetic-LEGO datasets are presented, demonstrating that SCANet significantly improves assembly performance compared to MEPNet without correction. Chamfer distance (CD) metrics are multiplied by a factor of 10^5.

Method	Pose Acc ↑ (%)		CR ↑ (%)	MPR ↓ (%)
	Component	Step		
MEPNet [3]	38.58	28.66	–	–
MEPNet*	42.04	32.48	14.63	9.74
SCANet (w.o. AR)	48.78	36.10	19.23	8.44
SCANet (w.o. IM)	45.99	33.17	**23.40**	18.58
SCANet (w.o. 6D&IM)	35.80	28.46	14.54	23.25
SCANet	**49.31**	**36.96**	19.31	**6.35**

TABLE II: Ablation experiments of SCANet structure on LEGO-ECA. 'MEPNet*' denotes our adapted MEPNet for error correction task. '(w.o. AR)' indicates the absence of the assembly result branch in the convolutional neural network backbone. '(w.o. IM)' signifies the removal of the 2D image encoder from the component pose encoder. '(w.o. 6D&IM)' denotes the removal of both the 2D image encoder of the component pose encoder and the 6D pose encoder.

correctness, it might introduce components that are still incorrect as input for the next step (as illustrated in Figure 6 (d)). Additionally, to further validate the generalization capability of our method, we randomly select another 150 assembly manuals from the Synthetic LEGO dataset for assembly error correction experiments. It is important to note that these 150 test manuals have no overlap with the manuals included in the LEGO-ECA dataset.

C. Evaluation Metrics

To evaluate the performance of SCANet on the single-step assembly error correction task, we employed various metrics. Firstly, we referred to some evaluation metrics from MEPNet, including the component pose accuracy after correction, *Chamfer distance* [25], and step-wise pose accuracy (considering a step correct if all component predictions in that step are correct). Secondly, as we adopted the setwise setting from MEPNet, we computed the normalized *Mistakes to Complete (MTC)* score [1]. Lastly, we introduced two additional metrics to assess the correction effectiveness, namely *Correction Rate* (CR) and *Misplacement Rate* (MPR).

We define the Correction Rate as the ratio of the number of originally incorrect components corrected to the correct pose to the total number of originally incorrect components, formulated as follows:

$$CR = \frac{N_c}{N_{ic}} \tag{3}$$

Here, N_c represents the number of originally incorrect components corrected to the correct pose, and N_{ic} denotes the total number of originally incorrect components.

We define the Misplacement Rate as the ratio of the number of originally correct components corrected to the wrong pose to the total number of originally correct components, expressed as:

$$MPR = \frac{N_w}{N_c} \tag{4}$$

Here, N_w represents the number of originally correct components corrected to the wrong pose, and N_c indicates the total number of originally correct components.

D. Results and Analysis

Assembly Error Correction Experiments. We conducted single-step assembly error correction experiments on the test sets from the LEGO-ECA dataset and a test set extracted from the Synthetic LEGO dataset. As shown in Table I, the overall assembly performance significantly improves when using SCANet to correct the assembly results produced by MEPNet. For the LEGO-ECA test set, we observe relatively low assembly accuracy of MEPNet during consecutive assembly steps. However, after correction by SCANet, the accuracy of MEPNet's assembly results improves significantly. This result validates the ability of our SCANet to correct incorrectly assembled components. Regarding the Synthetic-LEGO test set, which neither SCANet nor MEPNet had seen during training, SCANet is still able to correct the assembly results produced by MEPNet. This further demonstrates the generalization capability of our method, indicating its ability to correct errors in completely unseen components.

In the end, the visual results of the correction using SCANet are also presented in Figure 6. We can observe that SCANet effectively identifies and corrects erroneous components in the assembly results produced by the MEPNet. Particularly in Figure 6 (d), not only are there errors present in the current step, but multiple assembly errors are also in the already assembled parts (as 100% correction cannot be guaranteed). However, in situations that have not been encountered during training, SCANet can still perform error correction, further demonstrating the robustness of SCANet.

Ablation Experiments. In the ablation study, we first adapted MEPNet by replacing its component encoder with our component pose encoder and added an assembly result input branch to make it suitable for the single-step assembly error correction task. We then compared it with our SCANet. Next, we removed the branch responsible for inputting assembly results from the backbone of SCANet to assess the necessity of the assembly-shape branch. Then, we validated

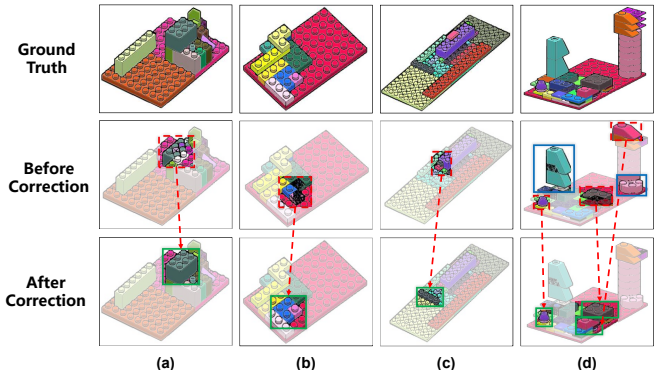


Fig. 6: The visualization results of correcting misassembled components using SCANet. **Red boxes** indicate components corrected before error correction, **green boxes** indicate components corrected after error correction, and **blue boxes** indicate assembly errors that could not be successfully corrected in previous steps (*i.e.*, accumulated assembly errors).

the role of the image encoder in the component pose encoder, as mentioned earlier, which can alleviate SCANet's problem of correcting components with the same shape but different 6D poses to the same pose. Finally, we further removed the 6D pose encoder from the component pose encoder, retaining only the 3D voxel encoder. However, before encoding the 3D voxel data, we performed an RT matrix transformation on the voxel data in advance to verify the effectiveness of the 6D pose encoder.

As depicted in Table II, although the improved version of MEPNet can correct misassembled results to some extent, its performance is significantly inferior to SCANet. Furthermore, we observed that after removing the 2D image encoder, while its correction rate (CR) is higher than that of the final SCANet, its misplacement rate (MPR) is also higher. As mentioned earlier, the absence of 2D image information causes the model to correct many components with the same shape but different poses to the same pose, thereby reducing the overall error correction capability. Finally, we observe that after removing the 6D pose encoder, the performance of SCANet drops sharply, highlighting the significant role played by this encoder.

VI. Conclusion and Future Works

This paper proposes a novel task, namely the single-step assembly error correction task, aimed at addressing how to handle errors in component assembly. Additionally, we construct the LEGO-ECA dataset based on Synthetic LEGO data specifically for this task. Furthermore, a novel framework called SCANet is introduced to tackle this task. Experimental results demonstrate that SCANet can identify and correct existing assembly errors, significantly improving assembly accuracy. For the future works, exploring better assembly error correction mechanisms to achieve improved correction results is warranted. Moreover, while we have focused on correcting components in the current step, the correction of components in non-current steps remains unexplored, presenting a more challenging and potentially valuable research direction.

References

[1] Z. Chen, K. Srinet, C. R. Qi, H. Fan, J. Ma, L. Zitnick, D. Guo, T. Xiao, S. Xie, X. Chen, A. Szlam, S. Tulsiani, H. Yu, and J. Gray, "Order-aware generative modeling using the 3d-craft dataset," in *ICCV*, 2019.

[2] H. Chung, J. Kim, B. Knyazev, J. Lee, G. W. Taylor, J. Park, and M. Cho, "Brick-by-brick: Combinatorial construction with deep reinforcement learning," in *NeurIPS*, 2021.

[3] R. Wang, Y. Zhang, J. Mao, C. Cheng, and J. Wu, "Translating a visual LEGO manual to a machine-executable plan," in *ECCV*, 2022.

[4] R. Wang, Y. Zhang, J. Mao, R. Zhang, C. Cheng, and J. Wu, "Ikea-manual: Seeing shape assembly step by step," in *NeurIPS*, 2022.

[5] X. Zhou and Q. Pham, "Can robots assemble an IKEA chair?" *Sci. Robotics*, vol. 3, no. 17, 2018.

[6] G. Zhan, Q. Fan, K. Mo, L. Shao, B. Chen, L. J. Guibas, and H. Dong, "Generative 3d part assembly via dynamic graph learning," in *NeurIPS*, 2020.

[7] Y. Li, K. Mo, L. Shao, M. Sung, and L. J. Guibas, "Learning 3d part assembly from a single image," in *ECCV*, 2020.

[8] Z. Wang, S.-W. Zhang, N. Wang, J.-Y. Xu, and D.-J. Cheng, "A feature-based assembly information modeling method for complex products' 3d assembly design," *Proc Inst Mech Eng B J Eng Manuf*, vol. 237, no. 9, 2023.

[9] R. Wu, C. Tie, Y. Du, Y. Zhao, and H. Dong, "Leveraging se (3) equivariance for learning 3d geometric shape assembly," in *ICCV*, 2023.

[10] P. S. Ogun, Z. Usman, K. Dharmaraj, and M. R. Jackson, "3d vision assisted flexible robotic assembly of machine components," in *ICMV*, 2015.

[11] C.-H. Shen, H. Fu, K. Chen, and S.-M. Hu, "Structure recovery by part assembly," *TOG*, vol. 31, no. 6, 2012.

[12] X. Xie, K. Xu, N. J. Mitra, D. Cohen-Or, W. Gong, Q. Su, and B. Chen, "Sketch-to-design: Context-based part assembly," in *Comput Graph Forum*, 2013.

[13] K. Mo, S. Zhu, A. X. Chang, L. Yi, S. Tripathi, L. J. Guibas, and H. Su, "Partnet: A large-scale benchmark for fine-grained and hierarchical part-level 3d object understanding," in *CVPR*, 2019.

[14] H. Zheng, R. Lee, and Y. Lu, "Ha-vid: A human assembly video dataset for comprehensive assembly knowledge understanding," *ISO4*, vol. 36, 2024.

[15] B. Jones, D. Hildreth, D. Chen, I. Baran, V. G. Kim, and A. Schulz, "Automate: A dataset and learning approach for automatic mating of cad assemblies," *ACM Trans. Graph*, vol. 40, 2021.

[16] Y. Chen, H. Li, D. Turpin, A. Jacobson, and A. Garg, "Neural shape mating: Self-supervised object assembly with adversarial shape priors," in *CVPR*, 2022, pp. 12 714–12 723.

[17] X. Liu, J. Zhang, R. Hu, H. Huang, H. Wang, and L. Yi, "Self-supervised category-level articulated object pose estimation with part-level SE(3) equivariance," in *ICLR*, 2023.

[18] A. N. Harish, R. Nagar, and S. Raman, "RGL-NET: A recurrent graph learning framework for progressive part assembly," in *WACV*, 2022, pp. 647–656.

[19] Y. Lee, E. S. Hu, and J. J. Lim, "IKEA furniture assembly environment for long-horizon complex manipulation tasks," in *ICRA*, 2021, pp. 6343–6349.

[20] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *ECCV*, 2020, pp. 213–229.

[21] A. Newell, K. Yang, and J. Deng, "Stacked hourglass networks for human pose estimation," in *ECCV*, 2016, pp. 483–499.

[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," vol. 30, 2017.

[23] M. Kocabas, S. Karagoz, and E. Akbas, "Multiposenet: Fast multi-person pose estimation using pose residual network," in *ECCV*, 2018.

[24] Y. Li, S. Yang, P. Liu, S. Zhang, Y. Wang, Z. Wang, W. Yang, and S.-T. Xia, "Simcc: A simple coordinate classification perspective for human pose estimation," in *ECCV*, 2022, pp. 89–106.

[25] H. Fan, H. Su, and L. J. Guibas, "A point set generation network for 3d object reconstruction from a single image," in *CVPR*, 2017, pp. 2463–2471.